Sig Circuit

개요

Sig Circuit은 ECDSA 서명 검증을 담당하는 회로입니다. 이 회로는 secp256k1과 secp256r1 두 가지 타원곡선을 지원하며, 트랜잭션 서명과 ecrecover 프리컴파일의 서명을 검증합니다. 서명 검증 과정에서 공개키 복구, 주소 계산, 그리고 서명의 수학적 정확성을 확인합니다.

주요 역할:

- ECDSA 서명 검증 (secp256k1, secp256r1)
- 공개키 복구 (ecrecover)
- 이더리움 주소 계산
- 서명 데이터의 RLC 계산
- Sig Table을 통한 검증 결과 제공
- Keccak Table 룩업을 통한 주소 검증

파일 위치:

- 메인 파일: `zkevm-circuits/src/sig_circuit.rs`
- ECDSA 검증: `zkevm-circuits/src/sig_circuit/ecdsa.rs`
- 유틸리티: `zkevm-circuits/src/sig_circuit/utils.rs`
- 테이블 정의: `zkevm-circuits/src/table.rs`

지원하는 타원곡선

secp256k1

- 이더리움 표준 서명 곡선
- 트랜잭션 서명에 사용
- ecrecover 프리컴파일에서 사용
- 최대 검증 개수: `MAX_NUM_SIG_K1 = 100`

secp256r1 (P-256)

- NIST P-256 곡선
- p256Verify 프리컴파일에서 사용
- 최대 검증 개수: `MAX_NUM_SIG_R1 = 15`

CRT 정수 분해

ECDSA 서명 검증에서 큰 정수(256비트)를 효율적으로 처리하기 위해 Chinese Remainder Theorem (CRT) 표현을 사용합니다.

Limb 구조

```
const LIMB_BITS: usize = 88;

const NUM_LIMBS: usize = 3;
```

256비트 정수를 3개의 limb로 분해:

- Limb 0: 88비트 (하위)
- Limb 1: 88비트 (중간)
- Limb 2: 80비트 (상위)

총: 88 + 88 + 80 = 256비트

CRTInteger 구조

```
pub struct CRTInteger<F> {
pub truncation: OverflowInteger<F>, // limbs로 분해된 정수
pub limbs: Vec<AssignedValue<F>>, // 각 limb의 할당된 값
pub native: AssignedValue<F>, // 네이티브 필드에서의 값
pub value: Value<BigUint>, // 실제 큰 정수 값
}
```

이 표현 방식은:

- 큰 정수 연산을 작은 limb 단위로 분해
- Range check를 limb별로 수행하여 효율성 향상
- 모듈러 연산을 CRT를 통해 최적화

ECDSA 서명 검증 프로세스

1. 서명 데이터 구조

```
pub struct SignData<Fq, Affine> {
pub signature: (Fq, Fq, u8), // (r, s, v)
pub pk: Affine, // 공개키 (x, y)
pub msg: Vec<u8>, // 원본 메시지
pub msg_hash: Fq, // 메시지 해시
}
```

2. 검증 단계

Step 1: 공개키 유효성 검증

```
let pk_is_valid = ecc_chip.is_on_curve_or_infinity::<Affine>(ctx, &pk_assigned)
gate.assert_is_const(ctx, &pk_is_valid, F::one());
```

공개키가 타원곡선 위의 점인지 확인합니다.

Step 2: 서명 값 로드

r, s, msg_hash를 CRTInteger로 로드:

```
let integer_r = fq_chip.load_private(ctx, FqChip::fe_to_witness(&Value::known(*
let integer_s = fq_chip.load_private(ctx, FqChip::fe_to_witness(&Value::known(*
let msg_hash = fq_chip.load_private(ctx, FqChip::fe_to_witness(&Value::known(*m
```

Step 3: ECDSA 수학적 검증

`ecdsa_verify_no_pubkey_check` 함수에서 수행:

수학적 원리:

```
u1 = msg_hash * s^(-1) mod n
u2 = r * s^(-1) mod n
R = u1 * G + u2 * PK
```

여기서:

- G: 생성자 점 (generator point)

- PK: 공개키
- n: 스칼라 필드의 모듈러스
- R: 복구된 점

검증 조건:

```
R.x mod n == r
```

Step 4: v 값 검증 (secp256k1만 해당)

v는 복구된 공개키의 y 좌표의 홀짝성을 나타냅니다.

```
// v는 0 또는 1
assert!(*v == 0 || *v == 1, "v is not boolean");


// 제약 조건: v + 2*tmp = y
// - v는 이진수
// - tmp는 87비트 미만
let  assigned_y_is_odd = gate.load_witness(ctx, Value::known(F::from(*v  as u64)
gate.assert_bit(ctx, assigned_y_is_odd);


let  y_tmp = (y_value - F::from(*v  as u64)) * F::TWO_INV;
let  assigned_y_tmp = gate.load_witness(ctx, Value::known(y_tmp));


// y_tmp_double = (y_value - y_last_bit)
let  y_tmp_double = gate.mul(ctx,
```

```
QuantumCell::Existing(assigned_y_tmp),
QuantumCell::Constant(F::from(2))
);

let  y_rec = gate.add(ctx,
QuantumCell::Existing(y_tmp_double),
QuantumCell::Existing(assigned_y_is_odd)
);

let  y_is_ok = gate.is_equal(ctx,
QuantumCell::Existing(*assigned_y_limb),
QuantumCell::Existing(y_rec)
);

// assigned_y_tmp는 87비트 범위 체크
ecc_chip.field_chip.range.range_check(ctx, &assigned_y_tmp, LIMB_BITS - 1);
```

이 검증은 v 값이 실제로 y 좌표의 최하위 비트와 일치하는지 확인합니다.

Step 5: 최종 검증 결과

```
let  sig_is_valid = gate.and_many(ctx, vec![
QuantumCell::Existing(sig_is_valid), // ECDSA 수학적 검증
QuantumCell::Existing(y_is_ok), // v 값 검증 (k1만)
```

```
QuantumCell::Existing(pk_not_zero), // 공개키가 (0,0)이 아님
]);
```

주의: 이 회로는 `sig_is_valid` 가 true임을 강제하지 않습니다. 호출자가 이 값을 확인해야 합니다.

공개키 복구 (ecrecover)

ecrecover 프로세스

1. **서명 값으로부터 점 복구**:

```
R = u1 * G + u2 * PK
```

2. **y 좌표 복구**:

- v 값을 사용하여 y의 홀짝성 결정
- x 좌표®로부터 y 계산: `y^2 = x^3 + 7 (mod p)`

3. **공개키 계산**:

```
PK = (R - u1 * G) / u2
```

주소 계산

공개키로부터 이더리움 주소를 계산:

```
// 1. 공개키를 바이트로 변환 (64바이트: x || y)
let  pk_le = pk_bytes_le_generic(&sign_data.pk);
let  pk_be = pk_bytes_swap_endianness(&pk_le);
```

```
// 2. Keccak256 해시 계산
let pk_hash = keccak256(pk_be);


// 3. 하위 20바이트를 주소로 사용
let address = ecdsa_chip.range.gate.inner_product(
ctx,
powers_of_256_cells[..20].to_vec(),
pk_hash_cells[..20].to_vec(),
);
```

주소 계산 공식:

```
address = keccak256(public_key)[12:32] // 마지막 20바이트
```

SignDataDecomposed 구조

서명 데이터를 바이트 단위로 분해하여 RLC 계산에 사용합니다.

```
pub struct SignDataDecomposed<F: Field> {
pub pk_hash_cells: Vec<QuantumCell<F>>, // 공개키 해시 (32바이트)
pub msg_hash_cells: Vec<QuantumCell<F>>, // 메시지 해시 (32바이트)
pub pk_cells: Vec<QuantumCell<F>>, // 공개키 (64바이트: y || x)
pub address: AssignedValue<F>, // 계산된 주소
```

```
pub is_address_zero: AssignedValue<F>, // 주소가 0인지 여부
pub r_cells: Vec<QuantumCell<F>>, // 서명 r (32바이트)
pub s_cells: Vec<QuantumCell<F>>, // 서명 s (32바이트)
}
```

바이트 분해 과정

1. CRTInteger를 바이트로 분해

```
fn assert_crt_int_byte_repr(
&self,
ctx: &mut Context<F>,
range_chip: &RangeConfig<F>,
crt_int: &CRTInteger<F>,
byte_repr: &[QuantumCell<F>],
powers_of_256: &[QuantumCell<F>],
) -> Result<(), Error>
```

검증 과정:

- Limb 0 (88비트): 바이트 0-10 (11바이트)
- Limb 1 (88비트): 바이트 11-21 (11바이트)
- Limb 2 (80비트): 바이트 22-31 (10바이트)

```
// Limb 1 복구 및 검증
let limb1_recover = flex_gate_chip.inner_product(
```

```
ctx,
byte_repr[0..11].to_vec(),
powers_of_256[0..11].to_vec(),
);
flex_gate_chip.assert_equal(
ctx,
QuantumCell::Existing(limb1_value),
QuantumCell::Existing(limb1_recover),
);
```

2. 공개키 바이트 배열

공개키는 y || x 순서로 64바이트:

```
let pk_x_le = x.to_repr().iter()
.map(|&x| QuantumCell::Witness(Value::known(F::from_u128(x as u128))))
.collect_vec();
let pk_y_le = y.to_repr().iter()
.map(|&y| QuantumCell::Witness(Value::known(F::from_u128(y as u128))))
.collect_vec();

let assigned_pk_le_selected = [pk_y_le, pk_x_le].concat(); // 64바이트
```

Random Linear Combination (RLC)

서명 데이터의 무결성을 효율적으로 검증하기 위해 RLC를 사용합니다.

Challenge 값

```
// EVM Word Challenge (메시지 해시, r, s용)
let evm_challenge_powers = iter::successors(
Some(Value::known(F::one())),
|coeff| Some(challenges.evm_word() * coeff)
).take(32);

// Keccak Input Challenge (공개키용)
let keccak_challenge_powers = iter::successors(
Some(Value::known(F::one())),
|coeff| Some(challenges.keccak_input() * coeff)
).take(64);
```

RLC 계산

1. 메시지 해시 RLC

```
let msg_hash_rlc = rlc_chip.gate.inner_product(
ctx,
sign_data_decomposed.msg_hash_cells.iter().take(32).cloned().collect_vec(),
evm_challenge_powers.clone(),
```

```
);
```

수식:

```
msg_hash_rlc = Σ(msg_hash[i] * challenge^i) for i = 0..31
```

2. 공개키 RLC

```
let  pk_rlc = rlc_chip.gate.inner_product(
ctx,
sign_data_decomposed.pk_cells.clone(), // 64바이트
keccak_challenge_powers,
);
```

3. 공개키 해시 RLC

```
let  pk_hash_rlc = rlc_chip.gate.inner_product(
ctx,
sign_data_decomposed.pk_hash_cells.clone(), // 32바이트
evm_challenge_powers.clone(),
);
```

4. 서명 r, s RLC

```
let  r_rlc = rlc_chip.gate.inner_product(
```

```
ctx,
sign_data_decomposed.r_cells.clone(),
evm_challenge_powers.clone(),
);


let s_rlc = rlc_chip.gate.inner_product(
ctx,
sign_data_decomposed.s_cells.clone(),
evm_challenge_powers,
);
```

Keccak Table 룩업

공개키 해시 검증을 위해 Keccak Table과 룩업을 수행합니다.

룩업 구조

```
fn enable_keccak_lookup(
&self,
config: &SigCircuitConfig<F>,
ctx: &mut Context<F>,
offset: usize,
is_address_zero: &AssignedValue<F>,
pk_rlc: &AssignedValue<F>,
```

```
pk_hash_rlc: &AssignedValue<F>,

) -> Result<(), Error>
```

레이아웃

```
| q_keccak | rlc |

| -------- | --------------- |

| 1 | is_address_zero |

| | pk_rlc |

| | pk_hash_rlc |
```

룩업 제약 조건

```
meta.lookup_any("keccak lookup table", |meta| {

let  q_keccak = meta.query_selector(q_keccak);

let  is_address_zero = meta.query_advice(rlc_column, Rotation::cur());

let  is_enable = q_keccak * not::expr(is_address_zero);


let  input = [

is_enable.clone(), // q_enable

is_enable.clone(), // is_final

is_enable.clone() * meta.query_advice(rlc_column, Rotation(1)), // input_rlc (pk

is_enable.clone() * 64usize.expr(), // input_len (64바이트)
```

```
is_enable * meta.query_advice(rlc_column, Rotation(2)), // output_rlc (pk_hash_r

];

let  table = [

meta.query_fixed(keccak_table.q_enable, Rotation::cur()),

meta.query_advice(keccak_table.is_final, Rotation::cur()),

meta.query_advice(keccak_table.input_rlc, Rotation::cur()),

meta.query_advice(keccak_table.input_len, Rotation::cur()),

meta.query_advice(keccak_table.output_rlc, Rotation::cur()),

];



input.into_iter().zip(table).collect()

});
```

검증 내용:

- `pk_rlc` 를 입력으로 Keccak256 해시 계산
- 출력이 `pk_hash_rlc` 와 일치하는지 확인
- 주소가 0인 경우 룩업 비활성화 (`is_address_zero` 플래그)

Sig Table 노출

검증 결과를 Sig Table에 노출하여 다른 회로에서 사용할 수 있도록 합니다.

AssignedSignatureVerify 구조

```
pub  struct AssignedSignatureVerify<F: Field> {
```

```
pub address: AssignedValue<F>, // 복구된 주소
pub msg_len: usize, // 메시지 길이
pub msg_rlc: Value<F>, // 메시지 RLC
pub msg_hash_rlc: AssignedValue<F>, // 메시지 해시 RLC
pub r_rlc: AssignedValue<F>, // 서명 r RLC
pub s_rlc: AssignedValue<F>, // 서명 s RLC
pub v: AssignedValue<F>, // 서명 v 값
pub sig_is_valid: AssignedValue<F>, // 서명 유효성
}
```

Sig Table 할당

```
layouter.assign_region(
|| "expose sig table",
|mut region| {
for (idx, assigned_sig_verif) in assigned_sig_verifs.iter().enumerate() {
// q_enable 설정
region.assign_fixed(
|| "assign sig_table selector",
config.sig_table.q_enable,
idx,
|| Value::known(F::one()),
)?;
```

```
// 각 필드를 Sig Table에 복사
assigned_sig_verif.v.copy_advice(&mut  region, config.sig_table.sig_v, idx);
assigned_sig_verif.r_rlc.copy_advice(&mut  region, config.sig_table.sig_r_rlc, i
assigned_sig_verif.s_rlc.copy_advice(&mut  region, config.sig_table.sig_s_rlc, i
assigned_sig_verif.address.copy_advice(&mut  region, config.sig_table.recovered_
assigned_sig_verif.sig_is_valid.copy_advice(&mut  region, config.sig_table.is_va
assigned_sig_verif.msg_hash_rlc.copy_advice(&mut  region, config.sig_table.msg_h
}
Ok(())
},
)?;
```

Sig Table 스키마

컬럼	설명
`q_enable`	행 활성화 플래그
`sig_v`	서명 v 값 (0 또는 1)
`sig_r_rlc`	서명 r의 RLC
`sig_s_rlc`	서명 s의 RLC
`recovered_addr`	복구된 이더리움 주소
`is_valid`	서명 유효성 (0 또는 1)

회로 구성 (Configuration)

SigCircuitConfig

```
pub struct SigCircuitConfig<F: Field> {
ecdsa_k1_config: FpChipK1<F>, // secp256k1 칩
ecdsa_r1_config: FpChipR1<F>, // secp256r1 칩
rlc_column: Column<Advice>, // RLC 계산용 컬럼
q_keccak: Selector, // Keccak 룩업 셀렉터
keccak_table: KeccakTable, // Keccak 테이블 참조
sig_table: SigTable, // Sig 테이블 참조
}
```

파라미터

```
// Halo2-ecc 파라미터
const LIMB_BITS: usize = 88;
const NUM_LIMBS: usize = 3;
const LOG_TOTAL_NUM_ROWS: usize = 20; // 2^20 = 1,048,576 행

// 최대 서명 개수
const MAX_NUM_SIG_K1: usize = 100; // secp256k1
```

```
const MAX_NUM_SIG_R1: usize = 15; // secp256r1


// 리소스 사용량
const CELLS_PER_SIG: usize = 468591; // 서명당 셀 개수
const LOOKUP_CELLS_PER_SIG: usize = 64227; // 서명당 룩업 셀 개수
```

RangeConfig 설정

```
let  range = RangeConfig::<F>::configure(
meta,
RangeStrategy::Vertical,
&num_advice, // Advice 컬럼 개수
&num_lookup_advice, // Lookup advice 컬럼 개수
1, // Fixed 컬럼 개수
LOG_TOTAL_NUM_ROWS - 1, // Lookup bits
0, // Lookup context id
LOG_TOTAL_NUM_ROWS, // K (회로 크기)
);
```

Phase 구분

Single Phase 모드 (`onephase` feature)

```
#[cfg(feature = "onephase")]
```

```
let rlc_column = meta.advice_column_in(halo2_proofs::plonk::FirstPhase);
```

Multi Phase 모드 (기본)

```
#[cfg(not(feature = "onephase"))]
let rlc_column = meta.advice_column_in(halo2_proofs::plonk::SecondPhase);
```

Multi-phase를 사용하면:

- Phase 1: ECDSA 검증 및 데이터 분해
- Phase 2: RLC 계산 (challenge 값 사용)

할당 프로세스 (Assignment)

전체 흐름

```
fn assign(
&self,
config: &SigCircuitConfig<F>,
layouter: &mut impl Layouter<F>,
signatures_k1: &[SignData<Fq_K1, Secp256k1Affine>],
signatures_r1: &[SignData<Fq_R1, Secp256r1Affine>],
challenges: &Challenges<Value<F>>,
) -> Result<Vec<AssignedSignatureVerify<F>>, Error>
```

Step 1: ECDSA 검증

```
// secp256k1 서명 검증
let assigned_ecdsas_k1 = signatures_k1
.iter()
.chain(std::iter::repeat(&SignData::default()))
.take(self.max_verify_k1)
.map(|sign_data| self.assign_ecdsa_generic(&mut ctx, ecdsa_k1_chip, sign_data))
.collect::<Result<Vec<AssignedECDSA<F, FpChipK1<F>>>, Error>>()?;


// secp256r1 서명 검증
let assigned_ecdsas_r1 = signatures_r1
.iter()
.chain(std::iter::repeat(&SignData::<Fq_R1, Secp256r1Affine>::default()))
.take(self.max_verify_r1)
.map(|sign_data| self.assign_ecdsa_generic(&mut ctx, ecdsa_r1_chip, sign_data))
.collect::<Result<Vec<AssignedECDSA<F, FpChipR1<F>>>, Error>>()?;
```

패딩 처리: 실제 서명 개수가 최대 개수보다 적으면 기본값으로 패딩합니다.

Step 2: 데이터 분해

```
let sign_data_k1_decomposed = signatures_k1
.iter()
.chain(std::iter::repeat(&SignData::default()))
```

```
.take(self.max_verify_k1)
.zip_eq(assigned_ecdsas_k1.iter())
.map(|(sign_data, assigned_ecdsa)| {
self.sign_data_decomposition_generic(
&mut  ctx,
ecdsa_k1_chip,
sign_data,
assigned_ecdsa,
)
})
.collect::<Result<Vec<SignDataDecomposed<F>>, Error>>()?;
```

CRTInteger를 바이트 배열로 분해하고 주소를 계산합니다.

Step 3: Phase 전환 (Multi-phase 모드)

```
#[cfg(not(feature = "onephase"))]
{
// 현재 룩업 테이블 완료
ecdsa_k1_chip.finalize(&mut  ctx);
ctx.print_stats(&["ECDSA context"]);
// Phase 2로 전환
ctx.next_phase();
}
```

Phase 2에서는 challenge 값을 사용하여 RLC를 계산합니다.

Step 4: RLC 계산 및 서명 검증

```
let (assigned_keccak_values, assigned_sig_values): (

Vec<[AssignedValue<F>; 3]>,

Vec<AssignedSignatureVerify<F>>,

) = signatures_k1

.iter()

.chain(std::iter::repeat(&SignData::default()))

.take(self.max_verify_k1)

.zip_eq(assigned_ecdsas_k1.iter())

.zip_eq(sign_data_k1_decomposed.iter())

.map(|((sign_data, assigned_ecdsa), sign_data_decomp)| {

self.assign_sig_verify_generic(

&mut  ctx,

&ecdsa_k1_chip.range,

sign_data,

sign_data_decomp,

challenges,

assigned_ecdsa,

)

})

.collect::<Result<Vec<([AssignedValue<F>; 3], AssignedSignatureVerify<F>)>, Erro
```

```
.into_iter()

.unzip();
```

각 서명에 대해:

- RLC 계산 (msg_hash, pk, pk_hash, r, s)
- Keccak 룩업을 위한 값 준비
- 최종 검증 결과 생성

Step 5: Keccak 룩업 활성화

```
for (i, [is_address_zero, pk_rlc, pk_hash_rlc]) in  assigned_keccak_values.iter(

let  offset = i * 3;

self.enable_keccak_lookup(

config,

&mut  ctx,

offset,

is_address_zero,

pk_rlc,

pk_hash_rlc,

)?;

}
```

각 서명에 대해 3행씩 사용하여 Keccak 룩업을 설정합니다.

Step 6: 최종화

```
let  lookup_cells = ecdsa_k1_chip.finalize(&mut  ctx);

log::info!("total number of lookup cells: {}", lookup_cells);

ctx.print_stats(&["ECDSA context"]);
```

모든 상수를 fixed 컬럼에 할당하고 룩업 셀을 정리합니다.

ECDSA 검증 상세 (ecdsa_verify_no_pubkey_check)

함수 시그니처

```
pub(crate) fn  ecdsa_verify_no_pubkey_check<F: PrimeField, CF: PrimeField, SF: P

base_chip: &FpConfig<F, CF>,

ctx: &mut Context<F>,

pubkey: &EcPoint<F, <FpConfig<F, CF> as FieldChip<F>>::FieldPoint>,

r: &CRTInteger<F>,

s: &CRTInteger<F>,

msghash: &CRTInteger<F>,

var_window_bits: usize,

fixed_window_bits: usize,

) -> (AssignedValue<F>, AssignedValue<F>, CRTInteger<F>)

where
```

```
GA: CurveAffineExt<Base = CF, ScalarExt = SF>,
```

반환값:

- `AssignedValue<F>` : 서명 유효성 (0 또는 1)
- `AssignedValue<F>` : 공개키가 (0,0)인지 여부
- `CRTInteger<F>` : 복구된 y 좌표

검증 단계

1. 공개키 영점 검사

```
let  is_pubkey_x_zero = ecc_chip.field_chip().is_zero(ctx, &pubkey.x);
let  is_pubkey_y_zero = ecc_chip.field_chip().is_zero(ctx, &pubkey.y);
let  is_pubkey_zero = ecc_chip.field_chip().range().gate().and(
ctx,
Existing(is_pubkey_x_zero),
Existing(is_pubkey_y_zero),
);
```

2. r, s 범위 검사

```
// r이 [1, n-1] 범위인지 확인
let  r_is_zero = scalar_chip.is_soft_zero(ctx, r);
let  r_in_range = scalar_chip.is_soft_nonzero(ctx, r);
let  r_is_valid = base_chip.range().gate().or(ctx, Existing(r_is_zero), Existing
```

```
// s가 [1, n-1] 범위인지 확인
let s_is_zero = scalar_chip.is_soft_zero(ctx, s);
let s_in_range = scalar_chip.is_soft_nonzero(ctx, s);
let s_is_valid = base_chip.range().gate().or(ctx, Existing(s_is_zero), Existing
```

3. u1, u2 계산

```
// u1 = msg_hash * s^(-1) mod n
let s_prime = scalar_chip.select(ctx, &one, s, &s_is_zero);
let u1 = scalar_chip.divide(ctx, msghash, &s_prime);
let u1 = scalar_chip.select(ctx, &zero, &u1, &s_is_zero);

// u2 = r * s^(-1) mod n
let u2 = scalar_chip.divide(ctx, r, &s_prime);
let u2 = scalar_chip.select(ctx, &zero, &u2, &s_is_zero);
```

엣지 케이스 처리: s가 0이면 u1, u2를 0으로 설정합니다.

4. 스칼라 곱셈

```
// u1 * G (고정 베이스 스칼라 곱셈)
let u1_is_zero = scalar_chip.is_zero(ctx, &u1);
let u1_prime = scalar_chip.select(ctx, &one, &u1, &u1_is_zero);
let u1_mul_affine = fixed_base::scalar_multiply::<F, _, GA>(
```

```
base_chip,
ctx,
&GA::generator(),
&u1_prime.truncation.limbs,
base_chip.limb_bits,
fixed_window_bits,
);
let  u1_mul = ecc_chip.select(ctx, &point_at_infinity, &u1_mul_affine, &u1_is_ze


// u2 * PK (가변 베이스 스칼라 곱셈)
let  u2_prime = scalar_chip.select(ctx, &one, &u2, &s_is_zero);
let  pubkey_prime = ecc_chip.load_random_point::<GA>(ctx);
let  pubkey_prime = ecc_chip.select(ctx, &pubkey_prime, pubkey, &is_pubkey_zero)


let  u2_mul_affine = scalar_multiply::<F, _, GA>(
base_chip,
ctx,
&pubkey_prime,
&u2_prime.truncation.limbs,
base_chip.limb_bits,
var_window_bits,
);
let  u2_is_zero = base_chip.range().gate().or(ctx, Existing(s_is_zero), Existing
```

```
let u2_mul = ecc_chip.select(ctx, &point_at_infinity, &u2_mul_affine, &u2_is_ze
```

5. 점 덧셈 (u1G + u2PK)

```
// lambda = (y2 - y1) / (x2 - x1)
let dx = base_chip.sub_no_carry(ctx, u2_mul.x(), u1_mul.x());
let dy = base_chip.sub_no_carry(ctx, u2_mul.y(), u1_mul.y());

let lambda = {
let a_val = base_chip.get_assigned_value(&dy);
let b_val = base_chip.get_assigned_value(&dx);
let b_inv = b_val.map(|bv| bv.invert().unwrap_or(CF::ZERO));
let quot_val = a_val.zip(b_inv).map(|(a, bi)| a * bi);
let quot = base_chip.load_private(ctx, FpConfig::<F, CF>::fe_to_witness(&quot_v
// quot * dx - dy = 0 mod p
let quot_b = base_chip.mul_no_carry(ctx, &quot, &dx);
let quot_constraint = base_chip.sub_no_carry(ctx, &quot_b, &dy);
base_chip.check_carry_mod_to_zero(ctx, &quot_constraint);
quot
};

// x3 = lambda^2 - x1 - x2
let lambda_sq = base_chip.mul_no_carry(ctx, &lambda, &lambda);
```

```
let lambda_sq_minus_px = base_chip.sub_no_carry(ctx, &lambda_sq, u1_mul.x());
let x_3_no_carry = base_chip.sub_no_carry(ctx, &lambda_sq_minus_px, u2_mul.x())
let x_3 = base_chip.carry_mod(ctx, &x_3_no_carry);


// y3 = lambda * (x1 - x3) - y1
let dx_13 = base_chip.sub_no_carry(ctx, u1_mul.x(), &x_3);
let lambda_dx_13 = base_chip.mul_no_carry(ctx, &lambda, &dx_13);
let y_3_no_carry = base_chip.sub_no_carry(ctx, &lambda_dx_13, u1_mul.y());
let y_3 = base_chip.carry_mod(ctx, &y_3_no_carry);
```

6. 무한원점 검사

```
// u1*G + u2*PK == 무한원점인 경우:
// 1. u1 == 0 AND u2 == 0
// 2. u1_mul.x == u2_mul.x AND u1_mul.y == -u2_mul.y


let u1_u2_are_zero = base_chip.range().gate().and(
ctx,
Existing(u1_is_zero),
Existing(u2_is_zero)
);


let u1_u2_x_eq = base_chip.is_equal(ctx, u1_mul.x(), u2_mul.x());
```

```
let u2_y_neg = base_chip.negate(ctx, u2_mul.y());
let u1_u2_y_neg = base_chip.is_equal(ctx, u1_mul.y(), &u2_y_neg);


let sum_is_infinity = base_chip.range().gate().or_and(
ctx,
Existing(u1_u2_are_zero),
Existing(u1_u2_x_eq),
Existing(u1_u2_y_neg),
);


let sum_is_not_infinity = base_chip.gate().not(ctx, QuantumCell::Existing(sum_i
```

7. 엣지 케이스 처리

```
// u1이 0이면 결과는 u2_mul
let x_3 = base_chip.select(ctx, u2_mul.x(), &x_3, &u1_is_zero);
let y_3 = base_chip.select(ctx, u2_mul.y(), &y_3, &u1_is_zero);


// u2가 0이면 결과는 u1_mul
let x_3 = base_chip.select(ctx, u1_mul.x(), &x_3, &u2_is_zero);
let y_3 = base_chip.select(ctx, u1_mul.y(), &y_3, &u2_is_zero);


// 합이 무한원점이면 (0, 0)
```

```
let  x_3 = base_chip.select(ctx, &zero, &x_3, &sum_is_infinity);

let  y_3 = base_chip.select(ctx, &zero, &y_3, &sum_is_infinity);
```

8. 최종 검증

```
// x_3 < n 확인

scalar_chip.enforce_less_than(ctx, &x_3);


// x_3 == r 확인

let  equal_check = base_chip.is_equal(ctx, &x_3, r);


// u1, u2 < n 확인

let  u1_small = big_less_than::assign::<F>(

base_chip.range(),

ctx,

&u1.truncation,

&n.truncation,

base_chip.limb_bits,

base_chip.limb_bases[1],

);

let  u2_small = big_less_than::assign::<F>(

base_chip.range(),

ctx,

&u2.truncation,
```

```
&n.truncation,
base_chip.limb_bits,
base_chip.limb_bases[1],
);


// 모든 조건 확인
let  res = base_chip.range().gate().and_many(
ctx,
vec![
Existing(r_is_valid), // r ∈ [0, n-1]
Existing(s_is_valid), // s ∈ [0, n-1]
Existing(u1_small), // u1 < n
Existing(u2_small), // u2 < n
Existing(sum_is_not_infinity), // u1*G + u2*PK ≠ ∞
Existing(equal_check), // x_3 == r
Existing(is_pubkey_not_zero), // PK ≠ (0,0)
],
);
```

secp256k1 vs secp256r1 차이점

secp256k1 (Ethereum 표준)

곡선 방정식: `y^2 = x^3 + 7 (mod p)`

파라미터:

- p (베이스 필드): `2^256 - 2^32 - 977`
- n (스칼라 필드): `FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141`
- a = 0, b = 7

특징:

- v 값 검증 필요 (y 좌표 홀짝성)
- 트랜잭션 서명에 사용
- ecrecover 프리컴파일에서 사용

secp256r1 (NIST P-256)

곡선 방정식: `y^2 = x^3 - 3x + b (mod p)`

파라미터:

- p (베이스 필드): `2^256 - 2^224 + 2^192 + 2^96 - 1`
- n (스칼라 필드): NIST 표준 값
- a = -3, b = 특정 상수

특징:

- v 값 검증 불필요 (공개키 (x, y) 직접 제공)
- p256Verify 프리컴파일에서 사용
- 웹 표준 및 하드웨어 지원

구현 차이

```
fn  is_k1_sig<Fp: PrimeField, Affine: CurveAffine<Base = Fp>>(&self) -> bool {
Affine::a() == Fp::ZERO // secp256k1은 a = 0
}


let (sig_is_valid, assigned_y_is_odd) = if  self.is_k1_sig::<Fp, Affine>() {
// secp256k1: v 값 검증 수행
let (y_is_ok, assigned_y_is_odd) =
self.check_y_oddness(ctx, ecdsa_chip, v, y_coord, pk_is_zero);
let  sig_is_valid = gate.and_many(ctx, vec![
QuantumCell::Existing(sig_is_valid),
QuantumCell::Existing(y_is_ok),
QuantumCell::Existing(pk_not_zero),
]);
(sig_is_valid, assigned_y_is_odd)
} else {
// secp256r1: v 값 검증 생략
let  sig_is_valid = gate.and_many(ctx, vec![
QuantumCell::Existing(sig_is_valid),
QuantumCell::Existing(pk_not_zero),
]);
(sig_is_valid, gate.load_zero(ctx))
};
```

최적화 기법

1. Vertical Cell Assignment

Sig Circuit은 halo2-lib의 vertical cell assignment 전략을 사용합니다.

```
// 셀을 수직으로 할당: 첫 번째 컬럼을 모두 사용한 후 다음 컬럼으로 이동
let row_num = 1 << LOG_TOTAL_NUM_ROWS; // 2^20 = 1,048,576
```

장점:

- 컬럼 간 균등한 셀 분배
- 효율적인 메모리 사용
- 동적 컬럼 개수 계산

2. 동적 컬럼 개수 계산

```
fn calc_required_advices(num_verif: usize) -> usize {
let mut num_adv = 1;
let total_cells = num_verif * CELLS_PER_SIG;
let row_num = 1 << LOG_TOTAL_NUM_ROWS;
while num_adv < COLUMN_NUM_LIMIT {
if num_adv * row_num > total_cells {
return num_adv;
}
num_adv += 1;
}
```

```
panic!("required advice columns exceeds limit");
}
```

서명 개수에 따라 필요한 advice 컬럼 개수를 자동으로 계산합니다.

3. 패딩 전략

```
signatures_k1
.iter()
.chain(std::iter::repeat(&SignData::default()))
.take(self.max_verify_k1)
```

실제 서명이 최대 개수보다 적으면 기본값으로 패딩하여 회로 크기를 일정하게 유지합니다.

4. 공유 RangeConfig

```
// secp256k1과 secp256r1이 하나의 RangeConfig 공유
let  ecdsa_k1_config = FpConfig::construct(range.clone(), LIMB_BITS, NUM_LIMBS,
let  ecdsa_r1_config = FpConfig::construct(range, LIMB_BITS, NUM_LIMBS, modulus:
```

두 곡선이 동일한 range check 테이블을 공유하여 리소스를 절약합니다.

5. 조기 종료 (Early Exit)

```
let  u1_is_zero = scalar_chip.is_zero(ctx, &u1);
```

```
let u1_mul = ecc_chip.select(ctx, &point_at_infinity, &u1_mul_affine, &u1_is_ze
```

u1 또는 u2가 0인 경우 불필요한 계산을 건너뜁니다.

사용 예시

블록에서 서명 데이터 생성

```
impl<F: Field> SubCircuit<F> for SigCircuit<F> {
fn new_from_block(block: &crate::witness::Block) -> Self {
SigCircuit {
max_verify_k1: MAX_NUM_SIG_K1,
max_verify_r1: MAX_NUM_SIG_R1,
signatures_k1: block.get_sign_data(true),
signatures_r1: block.get_sign_data_p256(true, MAX_NUM_SIG_R1),
_marker: Default::default(),
}
}
}
```

최소 행 개수 계산

```
fn min_num_rows_block(block: &crate::witness::Block) -> (usize, usize) {
let row_num = if block.circuits_params.max_vertical_circuit_rows == 0 {
Self::min_num_rows()
```

```
} else {
block.circuits_params.max_vertical_circuit_rows
};

// secp256k1 서명 개수
let  ecdsa_verif_k1_count = block.txs.iter()
.filter(|tx| !tx.tx_type.is_l1_msg())
.count()
+ block.precompile_events.get_ecrecover_events().len();

// secp256r1 서명 개수
let  ecdsa_verif_r1_count = block.precompile_events.get_p256_verify_events().len

// 패딩 트랜잭션을 위해 1개 예약
let  max_num_verify_k1 = MAX_NUM_SIG_K1 - 1;
let  max_num_verify_r1 = MAX_NUM_SIG_R1 - 1;

// 사용률 계산
let  min_row_num = [
(row_num / max_num_verify_k1) * ecdsa_verif_k1_count,
(row_num / max_num_verify_r1) * ecdsa_verif_r1_count,
].into_iter().max().unwrap();
```

```
(min_row_num, row_num)

}
```

Tx Circuit과의 연동

Tx Circuit은 Sig Table을 룩업하여 서명 검증 결과를 확인합니다:

```
// Tx Circuit에서 Sig Table 룩업

meta.lookup_any("sig table", |meta| {

let  input = [

msg_hash_rlc,

sig_v,

sig_r_rlc,

sig_s_rlc,

];

let  table = [

sig_table.msg_hash_rlc,

sig_table.sig_v,

sig_table.sig_r_rlc,

sig_table.sig_s_rlc,

];

input.into_iter().zip(table).collect()

});
```

```
// 복구된 주소 확인
let recovered_addr = sig_table.recovered_addr;
let is_valid = sig_table.is_valid;
```

주요 제약 조건 요약

1. 공개키 유효성

```
pk ∈ Curve ∨ pk = (0, 0)
```

2. 서명 범위

```
0 ≤ r < n
0 ≤ s < n
```

3. ECDSA 방정식

```
u1 = msg_hash * s^(-1) mod n
u2 = r * s^(-1) mod n
R = u1 * G + u2 * PK
R.x mod n = r
```

4. v 값 검증 (secp256k1)

```
v ∈ {0, 1}

v = R.y mod 2
```

5. 주소 계산

```
address = keccak256(pk_x || pk_y)[12:32]
```

6. RLC 일관성

```
msg_hash_rlc = Σ(msg_hash[i] * challenge^i)

pk_rlc = Σ(pk[i] * challenge^i)

pk_hash_rlc = Σ(pk_hash[i] * challenge^i)

r_rlc = Σ(r[i] * challenge^i)

s_rlc = Σ(s[i] * challenge^i)
```

7. Keccak 룩업

```
keccak256(pk_rlc, 64) = pk_hash_rlc
```

8. CRT 정수 분해

```
value = limb0 + limb1 * 2^88 + limb2 * 2^176

limb0 < 2^88
```

```
limb1 < 2^88

limb2 < 2^80
```

관련 문서

- Tx Circuit - 트랜잭션 서명 검증
- Keccak Circuit - 주소 계산용 해시
- 테이블 시스템 - Sig Table 구조
- EVM Circuit - ecrecover 프리컴파일

참고 자료

- halo2-ecc ECDSA 구현
- secp256k1 곡선 명세
- NIST P-256 곡선 명세
- ECDSA 서명 알고리즘
- Chinese Remainder Theorem